```
    private readonly ExchangeRatesClient _typedClient;                ❷
    private readonly AppDbContext _dbContext;                         ❷
    public UpdateExchangeRatesJob(                                    ❷
        ILogger<UpdateExchangeRatesJob> logger,                       ❷
        ExchangeRatesClient typedClient,                              ❷
        AppDbContext dbContext)                                       ❷
    {                                                                 ❷
        _logger = logger;                                             ❷
        _typedClient = typedClient;                                   ❷
        _dbContext = dbContext;                                       ❷
    }                                                                 ❷

    public async Task Execute(IJobExecutionContext context)           ❸
    {
        _logger.LogInformation("Fetching latest rates");
        var latestRates = await _typedClient.GetLatestRatesAsync();   ❹

        _dbContext.Add(latestRates);                                  ❺
        await _dbContext.SaveChangesAsync();                          ❺

        _logger.LogInformation("Latest rates updated");
    }
}
```

❶ Quartz.NET jobs must implement the IJob interface.

❷ You can use standard DI to inject any dependencies.

❸ IJob requires you to implement a single asynchronous method, Execute.

❹ Downloads the rates from the remote API

❺ Saves the rates to the database

Functionally, the `IJob` in listing 34.11 is doing a similar task to the `BackgroundService` implementation in listing 34.5, with a few notable exceptions:

- *The* `IJob` *defines only the task to execute; it doesn't define timing information*. In the `BackgroundService` implementation, we also had to control how often the task was executed.